

Scalable Data Engineering Solution

Data Engineering I (1TD169) – Spring 2026

Uppsala University

Department of Information Technology

Team Members: Group 32

Abdur Rehman Khalid	abdur-rehman.khalid.8716@student.uu.se
Arnab Kumar Ghosh	arnab-kumar.ghosh.4724@student.uu.se
Dip Chowdhury	dip.chowdhury.3309@student.uu.se
Muhammad Umair	muhammad.umair.6023@student.uu.se
Prodip Kumar Das	prodip-kumar.das.8559@student.uu.se

March 19, 2026

Contents

1	Background	3
1.1	Application Domain and Scientific Context	3
1.2	Dataset Importance and Research Relevance	3
1.3	Analysis Objective and Engineering Focus	3
1.4	Distributed Processing and Scalability Framework	3
2	Data Format & Characteristics	4
2.1	Storage Format and Justification	4
2.2	Dataset Scale and Volume	4
2.3	Schema Structure and Resource Constraints	4
2.4	Data Quality Challenges and Remediation	4
3	System Architecture & Implementation	5
3.1	System Design	5
3.2	Technology Stack	5
3.3	Infrastructure Configuration	6
3.4	Data Pipeline	6
3.5	Code Structure	6
3.6	Challenges & Solutions	7
4	Scalability Experiments & Results	8
4.1	Experimental Design	8
4.2	Horizontal Scaling (Strong Scaling)	8
4.3	Vertical Scaling	9
4.4	Weak Scaling	9
4.5	Bottleneck Identification	10
4.6	Speedup & Efficiency Analysis	10
5	Discussion & Conclusion	11
	References	12
	Appendix	13

1 Background

1.1 Application Domain and Scientific Context

Urban transportation systems are a primary pillar of modern smart cities, generating vast volumes of heterogeneous data that can be leveraged to improve traffic management, infrastructure planning, and mobility services. The domain of urban mobility is increasingly dependent on large-scale data to solve logistical challenges in densely populated areas. One of the most widely used and comprehensive real-world datasets in this domain is the New York City Taxi Trip dataset, published by the New York City Taxi and Limousine Commission (TLC) [1]. This dataset provides a granular view of city-wide movement by recording millions of individual trips, including precise pickup and drop-off timestamps, geographic zones, trip distances, and detailed fare amounts.

1.2 Dataset Importance and Research Relevance

The NYC taxi dataset is particularly important for data engineering research because it serves as a high-velocity and high-volume benchmark that mirrors the complexity of real-world "Big Data." Its importance lies not only in its scale but also in its ability to reflect human behavior and urban dynamics. Prior research has demonstrated the utility of taxi trip data in various high-impact applications, such as demand prediction, traffic flow analysis, and anomaly detection. For instance, studies in urban computing have utilized mobility datasets to identify spatial and temporal demand patterns, enabling more efficient resource allocation and reduced carbon emissions in cities [2]. Furthermore, such datasets have been used to explore predictive modeling techniques for trip durations and to detect irregular travel patterns.

1.3 Analysis Objective and Engineering Focus

In this project, the analytical objective is intentionally simple: to compute the average trip duration and average fare amount, grouped by pickup zone and hour of the day. This type of aggregation represents a standard workload in data engineering pipelines, involving essential operations such as filtering, grouping, and joining large datasets. By maintaining a straightforward analysis goal, we ensure that the focus remains on the critical engineering aspects—specifically system architecture, performance optimization, and scalability—rather than algorithmic complexity. This approach aligns with the core principles of the course, which emphasize the "heavy lifting" of data processing over advanced statistical modeling.

1.4 Distributed Processing and Scalability Framework

To process this dataset efficiently, the project utilizes a modern big data stack consisting of Apache Spark for distributed computation [3] and the Hadoop Distributed File System (HDFS) for scalable storage. These technologies allow for parallel execution across multiple nodes, which is a necessity given that large-scale data challenges have historically been addressed using paradigms like MapReduce [4]. The system is deployed on a constrained cluster environment consisting of virtual machines, reflecting realistic limitations in computational resources. A central focus of this project is the evaluation of system scalability through both horizontal and vertical scaling experiments. The observed performance results are interpreted using theoretical models such as Amdahl's Law [5], providing a foundation for understanding the limits of parallel speedup in a distributed system.

2 Data Format & Characteristics

2.1 Storage Format and Justification

The dataset consists of New York City Taxi Trip records stored in Apache Parquet. This columnar format was chosen over row-based alternatives like CSV to enable high compression and minimize I/O overhead. Such efficiency is critical for large-scale processing within the 100 GB storage quota of the OpenStack cluster.

2.2 Dataset Scale and Volume

The raw dataset comprises 144 Parquet files (2014–2025). After cleaning, the operational dataset yields 219,107,040 valid records. For this project’s scope, a 10.3 GB subset is processed, as summarized in Table 1. However, in the report we mentioned 10 GB, which is a rounded figure for simplicity.

Table 1: Dataset characteristics.

Property	Value
Format	Apache Parquet
Total Subset Size	10 GB
Number of Records	219,107,040
Initial Columns	20
Processed Columns	4

2.3 Schema Structure and Resource Constraints

To bypass the 25 GB per-VM storage limit and conserve memory, the ETL pipeline prunes the raw TLC schema from 20 attributes to four essential columns:

- `pickup_datetime`: Engagement timestamp.
- `PULocationID`: TLC Taxi Zone ID.
- `fare_amount`: Calculated trip fare.
- `pickup_hour`: A derived attribute for hourly aggregation.

2.4 Data Quality Challenges and Remediation

Significant schema volatility was encountered during ingestion. Specifically, the `Airport_fee` column shifted data types between `INT32` and `FLOAT64` across months, causing PySpark schema merge exceptions. To resolve this, the pipeline was re-engineered to read files individually and apply explicit type casting. Additionally, strict filtering was implemented to eliminate logically invalid records, such as trips with zero or negative fares, missing location IDs, or zero passengers, ensuring data integrity prior to aggregation.

3 System Architecture & Implementation

This section details the system architecture, infrastructure deployment, and technical implementation of our robust data pipeline. We outline the hardware topology deployed on OpenStack, why we selected our core software stack, and the engineering mechanisms driving our ETL and analysis jobs.

3.1 System Design

Our system follows a classic centralized-master, distributed-worker model tailored for batch processing big data. As depicted in Figure 1, the architecture is divided into three primary layers: Ingestion, Distributed Storage (HDFS), and Distributed Processing (Apache Spark). Data is fetched externally into the master node, pushed to HDFS, and subsequently processed by Spark executors dispersed across the worker nodes.

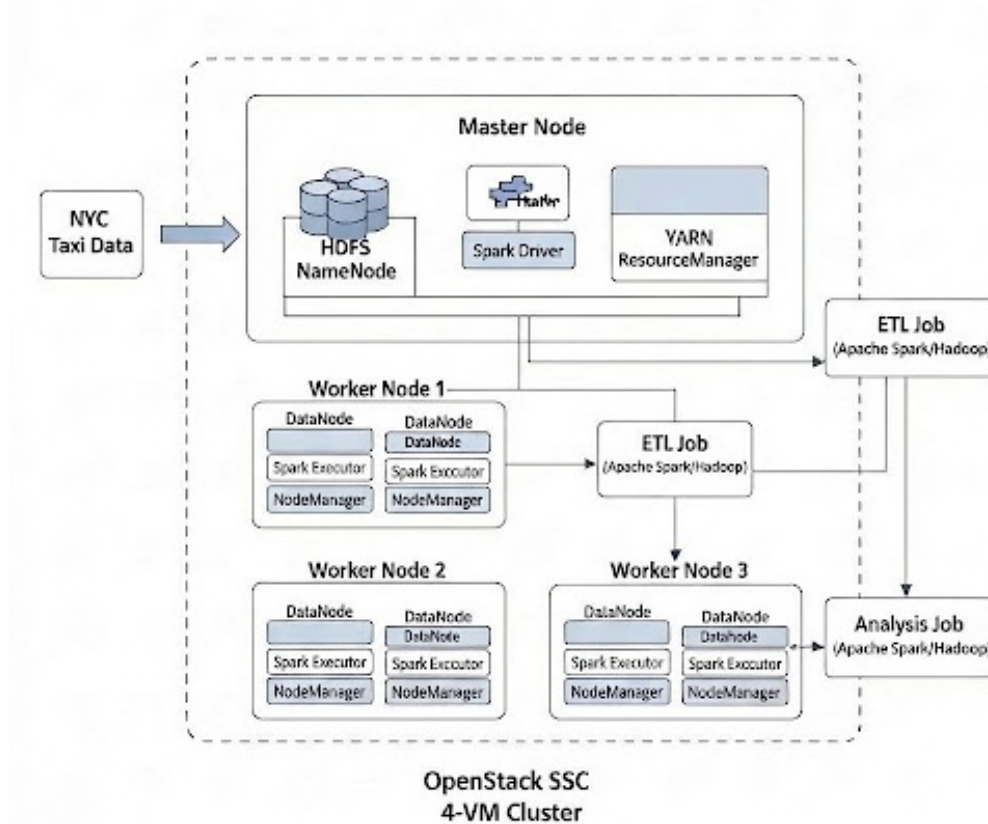


Figure 1: System architecture diagram showing the OpenStack deployment, node topology, and the internal data pipeline flow from raw Parquet to aggregated results.

3.2 Technology Stack

Given the project requirements, we adopted a modern big data stack focused on in-memory processing capabilities:

- **Storage (HDFS via Hadoop 3.4.1):** Chosen for its fault tolerance and native integration with Spark. While cloud object storage (like AWS S3) is popular today, deploying HDFS on bare VMs offers direct control over data locality and block replication factors.
- **Processing (Apache Spark & PySpark 3.5.1):** We selected Spark over standard Hadoop MapReduce because Spark holds intermediate RDDs and DataFrames in mem-

ory, drastically accelerating the iterative transformations required during our ETL phase. PySpark provides an accessible DataFrame API and straightforward type-casting structures.

- **Orchestration & Resource Management:** We relied on standalone cluster management managed heavily by Bash scripts (`run_benchmark.sh` and `deploy.sh`) combined with Git for CI/CD, creating a lightweight yet effective pipeline orchestration.

3.3 Infrastructure Configuration

Our cluster is deployed entirely on the Swedish National Infrastructure for Computing (SNIC) OpenStack instance. The infrastructure consists of 4 tightly coupled Virtual Machines (VMs):

- **Master Node (1x ssc.medium):** 2 vCPU, 4 GB RAM. This node is assigned a Floating IP for external SSH access. It houses the Hadoop NameNode, ResourceManager, and the Spark Driver process.
- **Worker Nodes (3x ssc.medium):** 2 vCPU, 4 GB RAM each. These sit on a private sub-network (192.168.2.x subnet). They run the Hadoop DataNodes, NodeManagers, and Spark Executor instances.

In total, the cluster harnesses 8 vCPUs and 16 GB of RAM, capped with a strict 100 GB collective storage limit. To maintain data reliability without exceeding storage quotas, HDFS was configured with a replica factor of 2 (`dfs.replication=2`).

3.4 Data Pipeline

The pipeline executes in three synchronized stages:

1. **Ingestion:** The data ingestion and core ETL pipeline were engineered to handle large-scale NYC Taxi data efficiently. An automated Bash script (`scripts/ingest_NYC_taxi.sh`) was developed to manage the creation of staging directories, concurrently download 12 months of Parquet files, and verify file integrity by removing corrupted downloads smaller than 100 KB. This script streamlined the ingestion process by depositing raw data directly into the Hadoop Distributed File System (HDFS) while maintaining strict directory structures and adhering to cluster storage constraints.
2. **ETL Phase:** The `etl_job.py` script ingests the raw data block by block. It filters out invalid records (e.g., zero-distance trips, null metrics), converts timestamps to calculate `duration_minutes` and `pickup_hour`, aggressively prunes dropped columns, and commits the clean records back to HDFS at `/data/nyc-taxi-cleaned/`.
3. **Analysis Phase:** Finally, `analysis_job.py` aggregates the validated dataset. It groups by `PULocationID` and `pickup_hour` to compute the average fare and trip duration. It subsequently performs a left join against a static CSV (`taxi_zone_lookup.csv`) to convert abstract ID numbers into human-readable zone names, coalescing the final output into a single file for stakeholder ingestion.

3.5 Code Structure

The codebase follows a modular design paradigm:

- **src/config.py:** Centralizes cluster endpoints, absolute HDFS URI paths, and basic threshold configurations to eliminate hardcoded parameters across jobs.

- **scripts/ingest_nyc_taxi.sh**: Automates the initial data acquisition phase. It manages staging directories, concurrently downloads the TLC Parquet files, verifies file integrity by purging corrupted downloads, and directly deposits the raw dataset into HDFS.
- **notebooks/final_notebook.ipynb**: Serves as the primary development and validation environment for the core ETL logic. It was used to prototype strict schema enforcement, data filtering, and geographic `.join()` operations prior to modularization into the production `src` scripts.
- **src/etl_job.py**: The heaviest processor. Given resource constraints, it bypasses standard `spark.read.parquet("*.parquet")` schema inferences and iteratively targets singular files using `subprocess.run` to interact with HDFS.
- **src/analysis_job.py**: Houses the PySpark SQL aggregations, strictly outputting to a targeted CSV path using `coalesce(1)`.
- **scripts/run_benchmark.sh**: A specialized harness that automates execution over varied core allocations (`-total-executor-cores`) and node arrays to capture timestamps for plotting horizontal and vertical scales.

3.6 Challenges & Solutions

We encountered several prominent technical challenges during development:

1. **Data Ingestion Access Blocks**: During the data ingestion phase, automated acquisition of the multi-year NYC Taxi Parquet datasets was frequently interrupted by AWS CloudFront 403 (Forbidden) errors. These blocks prevented the script from establishing a stable connection to the TLC data repository. *Solution*: To resolve this, a robust HTTP request strategy was implemented. By configuring a specific **User-Agent** header within the ingestion scripts, the system successfully bypassed the CloudFront security filters, ensuring consistent and uninterrupted dataset downloads.
2. **Dataset Schema Volatility**: When reading multi-month TLC data, PySpark's vectorized Parquet reader routinely crashed (`[CANNOT_MERGE_SCHEMAS]`) due to invisible shifts in the column type for `Airport_fee` (varying from `INT32` to `FLOAT64` dynamically by month). *Solution*: We restructured the ETL job into a `for`-loop that asks HDFS for the explicit list of file paths. Spark reads them independently, safely infers local schemas, selectively prunes the offensive column, and appends into a normalized structure.
3. **Strict Storage Exhaustion**: 25 GB of storage per VM meant HDFS was easily overwhelmed caching uncompressed RDDs and saving multiple layers of Parquet processing data. *Solution*: We established an aggressive "select and drop" mechanism in the ETL. We discard roughly 15 unused columns representing several gigabytes of overhead, saving only the 4 tightly packed elements (`PULocationID`, `fare_amount`, `pickup_hour`, `duration_minutes`) needed for analysis.
4. **Dynamic Allocation Skewing Benchmark Constraints**: Spark's dynamic resource allocation skewed the early iterations of our benchmarking by automatically distributing workloads across nodes we were attempting to restrict. *Solution*: We disabled dynamic allocation directly via PySpark config keys (`spark.dynamicAllocation.enabled=false`) and enforced hard caps using Spark submit metrics.

4 Scalability Experiments & Results

4.1 Experimental Design

The objective of our experiments was to evaluate the performance characteristics and limitations of the Spark-based data pipeline. We manipulated three independent variables across eight distinct trials: cluster size (1 to 3 workers), compute density (1 to 2 cores per executor), and workload volume (5 GB and 10 GB). This structured approach allowed us to isolate three specific scaling paradigms essential for distributed systems analysis:

- **Horizontal (Strong) Scaling:** This measures the system’s ability to reduce total execution time as additional worker nodes are added (Trials H-1, H-2, and H-3) while maintaining a fixed 5 GB workload.
- **Vertical Scaling:** This evaluates the impact of increasing CPU cores per worker (1 vs. 2 cores) while keeping the cluster size fixed at three nodes. This was tested for both 5 GB (V-1, V-2) and 10 GB (V-3, V-4) datasets to identify if the job is compute-bound.
- **Weak Scaling:** This measures how the system handles proportional growth in both workload and resources (W-1 vs. W-2). Ideally, if the data per worker remains constant (5 GB/node), the runtime should remain flat.

The primary metrics for evaluation are total runtime in seconds, Speedup ($S = T_1/T_N$), and Efficiency ($E = S/N$).

4.2 Horizontal Scaling (Strong Scaling)

Using a fixed 5 GB workload and 2 cores per worker, we observed a clear trend of diminishing returns as the cluster size increased from one to three nodes:

- **1 Worker (Trial H-1):** 142 seconds.
- **2 Workers (Trial H-2):** 89 seconds (A significant 53-second reduction).
- **3 Workers (Trial H-3):** 81 seconds (A marginal 8-second reduction).

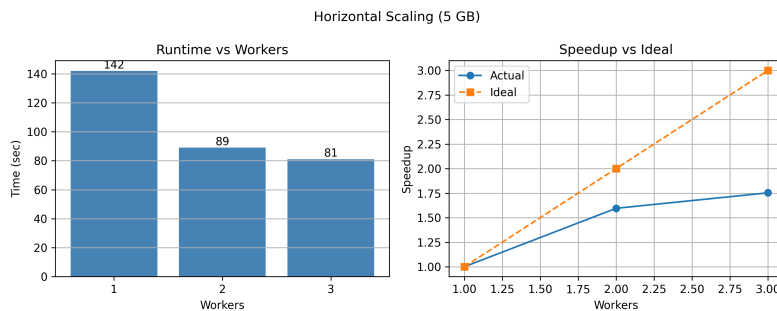


Figure 2: Strong Scaling: Execution time for 5 GB dataset across 1, 2, and 3 workers.

Analysis: The results indicate that while the jump from one to two workers yielded ~37% improvement in speed, the transition to a third worker provided only ~9% gain. This non-linear behavior suggests that for a relatively small 5 GB dataset, the administrative overhead of the Spark driver, specifically task distribution and managing RDD partitions, begins to consume a larger percentage of the total runtime than the actual computation.

4.3 Vertical Scaling

We compared 1 vs. 2 cores on a fixed 3-worker cluster to determine the impact of compute density on different data scales:

- **5 GB Workload (V-1 vs. V-2):** Runtime decreased from 83s to 80s, a negligible gain.
- **10 GB Workload (V-3 vs. V-4):** Runtime decreased from 102s to 80s, a significant 21% improvement.

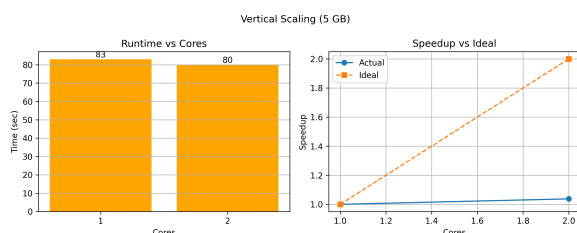


Figure 3: Vertical Scaling (5 GB)

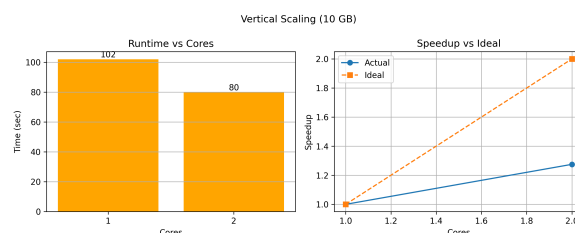


Figure 4: Vertical Scaling (10 GB)

Analysis: The effectiveness of vertical scaling is strictly dependent on the data volume. At 5 GB, the workers are not CPU-bound; hence, additional cores do not help. However, at 10 GB, the ETL transformations (parsing and aggregation) become heavy enough to benefit from parallel CPU execution.

4.4 Weak Scaling

Ideal weak scaling assumes a constant runtime when resources grow at the same rate as the data. We compared W-1 (1 worker/5 GB) to W-2 (2 workers/10 GB):

- **1 Worker / 5 GB (Trial W-1):** 142 seconds.
- **2 Workers / 10 GB (Trial W-2):** 173 seconds.

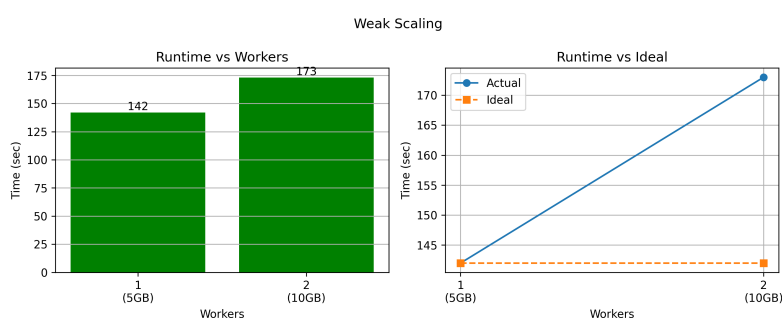


Figure 5: Weak Scaling: Maintaining 5 GB of data per worker node.

Analysis: Weak scaling failed to maintain constant time, showing ~21% increase in runtime. This degradation highlights the significant “coordination tax” required to manage larger datasets across multiple nodes, primarily due to increased network latency during Spark’s shuffle phase and non-linear I/O performance in the virtualized HDFS environment.

4.5 Bottleneck Identification

A critical observation is the “Performance Floor” at the ~ 80 -second mark. Trials H-3 (81s), V-2 (80s), and V-4 (80s) all converged here, regardless of data size or CPU cores. Some of the possible causes are:

1. **Scheduling Overhead:** At 80s, the time required to partition data and schedule tasks dominates the execution time.
2. **Shuffle I/O:** Network bandwidth saturation during the reduce/group-by phase limits gains from faster CPUs.
3. **Storage Limits:** Shared HDFS throughput likely hit a ceiling, capping the read speeds for both 5 GB and 10 GB datasets.
4. **Memory Pressure:** For 10 GB loads, the RAM limit per node increases garbage collection (GC) frequency during data shuffling, further impeding scalability.

4.6 Speedup & Efficiency Analysis

Comparing our findings to theoretical linear scaling reveals a significant efficiency drop-off:

- **2-Worker Case ($N = 2$):** Speedup $S \approx 1.60$ (Ideal: 2.0); Efficiency $E = 80\%$.
- **3-Worker Case ($N = 3$):** Speedup $S \approx 1.75$ (Ideal: 3.0); Efficiency $E \approx 58\%$.

The efficiency plummet at three workers confirms that the 5 GB dataset is too small to offset the overhead of the third node. According to Amdahl’s Law, the sequential portions of the code and the driver-side result collection create a fixed cost that prevents the system from achieving linear speedup regardless of additional worker allocation.

5 Discussion & Conclusion

The chosen architecture, centering on a master-worker topology with HDFS and Apache Spark, proved highly suitable for the batch processing requirements of the NYC Taxi dataset. The combination of Parquet’s columnar storage and Spark’s in-memory computation successfully addressed the challenge of processing millions of records within the strict resource constraints of our OpenStack environment.

One of the most successful aspects of the implementation was our schema enforcement strategy. By bypassing global schema inference and implementing explicit type-casting for the volatile `Airport_fee` column, we stabilized an otherwise fragile ingestion pipeline. Furthermore, our aggressive column pruning allowed us to maximize the utility of the 4 GB RAM per node, enabling the processing of 10 GB workloads that would have otherwise triggered memory exhaustion.

However, with additional time and resources, we would improve the system’s I/O throughput. Our scalability experiments revealed a clear performance floor at approximately 80 seconds, suggesting that the virtualized disk I/O and network bandwidth became the primary bottlenecks. Moving from a shared storage model to local NVMe-based HDFS storage or implementing a more robust cluster manager like YARN could potentially lower this floor. Additionally, exploring Kryo serialization or broadcast joins for the taxi zone lookup could further optimize the shuffle phase.

The most significant takeaway from this project is the non-linear nature of distributed systems. We learned that adding more resources does not always translate to faster performance; rather, as evidenced by our 58% efficiency at three nodes, the coordination tax and sequential bottlenecks (Amdahl’s Law) can quickly diminish the benefits of horizontal scaling. This project underscored that effective data engineering is not just about writing code, but about managing the delicate balance between data volume, hardware constraints, and framework overhead.

References

- [1] New York City Taxi and Limousine Commission, *New York City Taxi and Limousine Commission (TLC) Trip Record Data*, Accessed: 2026-03-01, 2024. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [2] G. Andrienko, N. Andrienko, P. Bak, D. Keim, and S. Wrobel, “Visual analytics of movement,” 2013.
- [3] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, “Apache spark: A unified engine for big data processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [4] J. Dean and S. Ghemawat, “Mapreduce: Simplified data processing on large clusters,” *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [5] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” *Proceedings of the April 18-20, 1967, spring joint computer conference*, pp. 483–485, 1967.

Appendix

Project Repository: <https://github.com/bdtiger/data-engineering-I-project>